

ASSIGNMENT-12.5

Name: D.BHARATH

H.T.No:2303A51027

Task Description #1 (Sorting – Merge Sort Implementation) • Task: Use AI to generate a Python program that implements the Merge Sort algorithm. • Instructions: o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order. o Ask AI to include time complexity and space complexity in the function docstring. o Verify the generated code with test cases.

Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

```
assg_12.5.py > ...
1  # generate a Python program that implements the Merge Sort algorithm. • Instructions: o Prompt AI to create a function merge_s
2  def merge_sort(arr):
3      """
4      Sorts a list in ascending order using the Merge Sort algorithm.
5
6      Time Complexity:  $O(n \log n)$ 
7      Space Complexity:  $O(n)$ 
8
9      Parameters:
10     arr (list): The list to be sorted.
11
12     Returns:
13     list: A new sorted list in ascending order.
14     """
15     if len(arr) <= 1:
16         return arr
17
18     mid = len(arr) // 2
19     left_half = merge_sort(arr[:mid])
20     right_half = merge_sort(arr[mid:])
21
22     return merge(left_half, right_half)
23
24 def merge(left, right):
25     sorted_list = []
26     left_index = right_index = 0
27
28     while left_index < len(left) and right_index < len(right):
29         if left[left_index] < right[right_index]:
30             sorted_list.append(left[left_index])
31             left_index += 1
32         else:
33             sorted_list.append(right[right_index])
34             right_index += 1
35
36     sorted_list.extend(left[left_index:])
37     sorted_list.extend(right[right_index:])
```

```

37     return sorted_list
38
39 # Test cases
40 if __name__ == "__main__":
41     test_cases = [
42         [38, 27, 43, 3, 9, 82, 10],
43         [5, 2, 9, 1, 5, 6],
44         [12, 11, 13, 5, 6, 7],
45         [],
46         [1],
47         [2, 1]
48     ]
49
50     for i, test in enumerate(test_cases):
51         print(f"Test case {i + 1}: {test}")
52         sorted_arr = merge_sort(test)
53         print(f"Sorted: {sorted_arr}\n")

```

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_12.5.py
Test case 1: [38, 27, 43, 3, 9, 82, 10]
Sorted: [3, 9, 10, 27, 38, 43, 82]

Test case 2: [5, 2, 9, 1, 5, 6]
Sorted: [1, 2, 5, 5, 6, 9]

Test case 3: [12, 11, 13, 5, 6, 7]
Sorted: [5, 6, 7, 11, 12, 13]

Test case 4: []
Sorted: []

Test case 5: [1]
Sorted: [1]

Test case 6: [2, 1]
Sorted: [1, 2]

```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - o Include docstrings explaining best, average, and worst-case complexities.
 - o Test with various inputs.
- Expected Output:
 - o Python code implementing binary search with AI-generated comments and docstrings.

```

def binary_search(arr, target):
    """
    Performs binary search to find the index of the target element in a sorted list.

    Best Case Complexity: O(1) - when the target is at the middle of the list.
    Average Case Complexity: O(log n) - when the target is found after several iterations.
    Worst Case Complexity: O(log n) - when the target is not found or is at the end of the list.

    Parameters:
    arr (list): A sorted list where the search is performed.
    target: The element to be searched for.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    return -1

# Test cases
if __name__ == "__main__":
    test_cases = [
        ([1, 2, 3, 4, 5], 3),
        ([1, 2, 3, 4, 5], 6),
    ]

```

```

    return 2

# Test cases
if __name__ == "__main__":
    test_cases = [
        ([1, 2, 3, 4, 5], 3),
        ([1, 2, 3, 4, 5], 6),
        ([1, 2, 3, 4, 5], 1),
        ([1, 2, 3, 4, 5], 5),
        ([], 1),
        ([1], 1),
        ([1], 0)
    ]

    for i, (arr, target) in enumerate(test_cases):
        result = binary_search(arr, target)
        print(f"Test case {i + 1}: Array: {arr}, Target: {target}, Result: {result}")

```

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_12.5.py
Test case 1: Array: [1, 2, 3, 4, 5], Target: 3, Result: 2
Test case 2: Array: [1, 2, 3, 4, 5], Target: 6, Result: -1
Test case 3: Array: [1, 2, 3, 4, 5], Target: 1, Result: 0
Test case 4: Array: [1, 2, 3, 4, 5], Target: 5, Result: 4
Test case 5: Array: [], Target: 1, Result: -1
Test case 6: Array: [1], Target: 1, Result: 0
Test case 7: Array: [1], Target: 0, Result: -1
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.

```

def search_appointment(appointments, appointment_id):
    """
    Searches for an appointment by its ID using linear search.

    Time Complexity: O(n) - where n is the number of appointments.

    Parameters:
    appointments (list): A list of appointment records (dictionaries).
    appointment_id: The ID of the appointment to search for.

    Returns:
    dict: The appointment record if found, otherwise None.
    """
    for appointment in appointments:
        if appointment['appointment_id'] == appointment_id:
            return appointment
    return None

def sort_appointments(appointments, key):
    """
    Sorts appointments based on a specified key (time or consultation fee) using the built-in sorted function.
    Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
    Parameters:
    appointments (list): A list of appointment records (dictionaries).
    key (str): The key to sort by ('time' or 'consultation_fee').
    Returns:
    list: A new list of sorted appointment records.
    """
    if key not in ['time', 'consultation_fee']:

```

```

    if key not in ['time', 'consultation_fee']:
        raise ValueError("Invalid key: must be 'time' or 'consultation_fee'.")

    return sorted(appointments, key=lambda x: x[key])

# Example usage
if __name__ == "__main__":
    appointments = [
        {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 10:00', 'consultation_fee': 100},
        {'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'consultation_fee': 150},
        {'appointment_id': 'A003', 'patient_name': 'Alice Smith', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 09:00', 'consultation_fee': 120}
    ]

    # Search for an appointment
    appointment = search_appointment(appointments, 'A002')
    print("Search Result:", appointment)

    # Sort appointments by time
    sorted_by_time = sort_appointments(appointments, key='time')
    print("Appointments sorted by time:", sorted_by_time)

    # Sort appointments by consultation fee
    sorted_by_fee = sort_appointments(appointments, key='consultation_fee')
    print("Appointments sorted by consultation fee:", sorted_by_fee)

```

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_12.5.py
Search Result: {'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'consultation_fee': 150}
Appointments sorted by time: [{'appointment_id': 'A003', 'patient_name': 'Alice Smith', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 09:00', 'consultation_fee': 120}, {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 10:00', 'consultation_fee': 100}, {'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'consultation_fee': 150}]
Appointments sorted by consultation fee: [{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 10:00', 'consultation_fee': 100}, {'appointment_id': 'A003', 'patient_name': 'Alice Smith', 'doctor_name': 'Dr. Smith', 'time': '2024-07-01 09:00', 'consultation_fee': 120}, {'appointment_id': 'A002', 'patient_name': 'Jane Doe', 'doctor_name': 'Dr. Brown', 'time': '2024-07-01 11:00', 'consultation_fee': 150}]
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task Description #4: Railway Ticket Reservation System Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

```

def search_ticket(bookings, ticket_id):
    """
    Searches for a ticket by its ID using linear search.

    Time Complexity: O(n) - where n is the number of bookings.

    Parameters:
    bookings (list): A list of booking records (dictionaries).
    ticket_id: The ID of the ticket to search for.

    Returns:
    dict: The booking record if found, otherwise None.
    """
    for booking in bookings:
        if booking['ticket_id'] == ticket_id:
            return booking
    return None

def sort_bookings(bookings, key):
    """Sorts bookings based on a specified key (travel date or seat number) using the built-in sorted function.
    Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
    Parameters:
    bookings (list): A list of booking records (dictionaries).
    key (str): The key to sort by ('travel_date' or 'seat_number').
    Returns:
    list: A new list of sorted booking records.
    """
    if key not in ['travel_date', 'seat_number']:
        raise ValueError("Invalid key: must be 'travel_date' or 'seat_number'.")

    return sorted(bookings, key=lambda x: x[key])

# Example usage
if __name__ == "__main__":
    bookings = [
        {'ticket_id': 'T001', 'passenger_name': 'John Doe', 'train_number': '123', 'seat_number': 'A1', 'travel_date': '2024-06-15'},
        {'ticket_id': 'T002', 'passenger_name': 'Jane Doe', 'train_number': '123', 'seat_number': 'A2', 'travel_date': '2024-06-15'},
        {'ticket_id': 'T003', 'passenger_name': 'Alice Smith', 'train_number': '124', 'seat_number': 'B1', 'travel_date': '2024-06-16'}
    ]

```

```

    {'ticket_id': 'T003', 'passenger_name': 'Alice Smith', 'train_number': '124', 'seat_number': 'B1', 'travel_date': '2024-07-01'}
]

# Search for a ticket
ticket = search_ticket(bookings, 'T002')
print("Search Result:", ticket)

# Sort bookings by travel date
sorted_by_date = sort_bookings(bookings, key='travel_date')
print("Bookings sorted by travel date:", sorted_by_date)

# Sort bookings by seat number
sorted_by_seat = sort_bookings(bookings, key='seat_number')
print("Bookings sorted by seat number:", sorted_by_seat)

```

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_12.5.py
Search Result: {'ticket_id': 'T002', 'passenger_name': 'Jane Doe', 'train_number': '123', 'seat_number': 'A2', 'travel_date': '2024-07-02'}
}
Bookings sorted by travel date: [{'ticket_id': 'T001', 'passenger_name': 'John Doe', 'train_number': '123', 'seat_number': 'A1', 'travel_date': '2024-07-01'}, {'ticket_id': 'T003', 'passenger_name': 'Alice Smith', 'train_number': '124', 'seat_number': 'B1', 'travel_date': '2024-07-01'}, {'ticket_id': 'T002', 'passenger_name': 'Jane Doe', 'train_number': '123', 'seat_number': 'A2', 'travel_date': '2024-07-02'}]
Bookings sorted by seat number: [{'ticket_id': 'T001', 'passenger_name': 'John Doe', 'train_number': '123', 'seat_number': 'A1', 'travel_date': '2024-07-01'}, {'ticket_id': 'T002', 'passenger_name': 'Jane Doe', 'train_number': '123', 'seat_number': 'A2', 'travel_date': '2024-07-02'}, {'ticket_id': 'T003', 'passenger_name': 'Alice Smith', 'train_number': '124', 'seat_number': 'B1', 'travel_date': '2024-07-01'}]
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.

```

def search_allocation(allocations, student_id):
    """
    Searches for a student's room allocation details by their ID using linear search.

    Time Complexity: O(n) - where n is the number of allocations.

    Parameters:
    allocations (list): A list of allocation records (dictionaries).
    student_id: The ID of the student to search for.

    Returns:
    dict: The allocation record if found, otherwise None.
    """
    for allocation in allocations:
        if allocation['student_id'] == student_id:
            return allocation
    return None

def sort_allocations(allocations, key):
    """Sorts allocations based on a specified key (room number or allocation date) using the built-in sorted function.
    Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().

    Parameters:
    allocations (list): A list of allocation records (dictionaries).
    key (str): The key to sort by ('room_number' or 'allocation_date').

    Returns:
    list: A new list of sorted allocation records.
    """
    if key not in ['room_number', 'allocation_date']:
        raise ValueError("Invalid key: must be 'room_number' or 'allocation_date'.")

    return sorted(allocations, key=lambda x: x[key])

# Example usage

```

```

# Example usage
if __name__ == "__main__":
    allocations = [
        {'student_id': 'S001', 'room_number': '101', 'floor': '1', 'allocation_date': '2024-07-01'},
        {'student_id': 'S002', 'room_number': '102', 'floor': '1', 'allocation_date': '2024-07-02'},
        {'student_id': 'S003', 'room_number': '201', 'floor': '2', 'allocation_date': '2024-07-01'}
    ]

    # Search for a student's allocation
    allocation = search_allocation(allocations, 'S002')
    print("Search Result:", allocation)

    # Sort allocations by room number
    sorted_by_room = sort_allocations(allocations, key='room_number')
    print("Allocations sorted by room number:", sorted_by_room)

    # Sort allocations by allocation date
    sorted_by_date = sort_allocations(allocations, key='allocation_date')
    print("Allocations sorted by allocation date:", sorted_by_date)

```

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_12.5.py
Search Result: {'student_id': 'S002', 'room_number': '102', 'floor': '1', 'allocation_date': '2024-07-02'}
Allocations sorted by room number: [{'student_id': 'S001', 'room_number': '101', 'floor': '1', 'allocation_date': '2024-07-01'}, {'student_id': 'S002', 'room_number': '102', 'floor': '1', 'allocation_date': '2024-07-02'}, {'student_id': 'S003', 'room_number': '201', 'floor': '2', 'allocation_date': '2024-07-01'}]
Allocations sorted by allocation date: [{'student_id': 'S001', 'room_number': '101', 'floor': '1', 'allocation_date': '2024-07-01'}, {'student_id': 'S003', 'room_number': '201', 'floor': '2', 'allocation_date': '2024-07-01'}, {'student_id': 'S002', 'room_number': '102', 'floor': '1', 'allocation_date': '2024-07-02'}]
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task Description #6: Online Movie Streaming Platform A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs

1. Search movies by movie ID.
 2. Sort movies based on rating or release year.
- Recommend searching and sorting algorithms using AI.
 - Justify the chosen algorithms.
 - Implement Python functions.


```

def search_movie(movies, movie_id):
    """
    Searches for a movie by its ID using linear search.

    Time Complexity: O(n) - where n is the number of movies.

    Parameters:
    movies (list): A list of movie records (dictionaries).
    movie_id: The ID of the movie to search for.

    Returns:
    dict: The movie record if found, otherwise None.
    """
    for movie in movies:
        if movie['movie_id'] == movie_id:
            return movie
    return None

def sort_movies(movies, key):
    """Sorts movies based on a specified key (rating or release year) using the built-in sorted function.
    Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
    Parameters:
    movies (list): A list of movie records (dictionaries).
    key (str): The key to sort by ('rating' or 'release_year').

    Returns:
    list: A new list of sorted movie records.
    """
    if key not in ['rating', 'release_year']:
        raise ValueError("Invalid key: must be 'rating' or 'release_year'.")

    return sorted(movies, key=lambda x: x[key])

# Example usage
if __name__ == "__main__":
    movies = [
        {'movie_id': 'M001', 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010},
        {'movie_id': 'M002', 'title': 'The Matrix', 'genre': 'Action', 'rating': 8.7, 'release_year': 1999},
    ]

```

```

# Search for a movie
movie = search_movie(movies, 'M002')
print("Search Result:", movie)

# Sort movies by rating
sorted_by_rating = sort_movies(movies, key='rating')
print("Movies sorted by rating:", sorted_by_rating)

# Sort movies by release year
sorted_by_year = sort_movies(movies, key='release_year')
print("Movies sorted by release year:", sorted_by_year)

```

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_12.5.py
Search Result: {'movie_id': 'M002', 'title': 'The Matrix', 'genre': 'Action', 'rating': 8.7, 'release_year': 1999}
Movies sorted by rating: [{'movie_id': 'M003', 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'release_year': 2014}, {'movie_id': 'M002', 'title': 'The Matrix', 'genre': 'Action', 'rating': 8.7, 'release_year': 1999}, {'movie_id': 'M001', 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010}]
Movies sorted by release year: [{'movie_id': 'M002', 'title': 'The Matrix', 'genre': 'Action', 'rating': 8.7, 'release_year': 1999}, {'movie_id': 'M001', 'title': 'Inception', 'genre': 'Sci-Fi', 'rating': 8.8, 'release_year': 2010}, {'movie_id': 'M003', 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'release_year': 2014}]
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task Description #7: Smart Agriculture Crop Monitoring System An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

- Use AI-assisted reasoning to select algorithms.
- Justify algorithm suitability.
- Implement searching and sorting in Python.

```

def search_crop(crops, crop_id):
    """
    Searches for crop details by crop ID using linear search.

    Time Complexity: O(n) - where n is the number of crops.

    Parameters:
    crops (list): A list of crop records (dictionaries).
    crop_id: The ID of the crop to search for.

    Returns:
    dict: The crop record if found, otherwise None.
    """
    for crop in crops:
        if crop['crop_id'] == crop_id:
            return crop
    return None

def sort_crops(crops, key):
    """Sorts crops based on a specified key (soil moisture level or yield estimate) using the built-in sorted function.
    Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
    Parameters:
    crops (list): A list of crop records (dictionaries).
    key (str): The key to sort by ('soil_moisture_level' or 'yield_estimate').

    Returns:
    list: A new list of sorted crop records.
    """
    if key not in ['soil_moisture_level', 'yield_estimate']:
        raise ValueError("Invalid key: must be 'soil_moisture_level' or 'yield_estimate'.")

    return sorted(crops, key=lambda x: x[key])

# Example usage
if __name__ == "__main__":
    crops = [
        {'crop_id': 'C001', 'crop_name': 'wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000},
        {'crop_id': 'C002', 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500},
    ]

```

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_12.5.py
Search Result: {'crop_id': 'C002', 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}
Crops sorted by moisture level: [{'crop_id': 'C001', 'crop_name': 'wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}, {'crop_id': 'C002', 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}, {'crop_id': 'C003', 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 2000}]
Crops sorted by yield estimate: [{'crop_id': 'C001', 'crop_name': 'wheat', 'soil_moisture_level': 30, 'temperature': 25, 'yield_estimate': 1000}, {'crop_id': 'C002', 'crop_name': 'Corn', 'soil_moisture_level': 40, 'temperature': 28, 'yield_estimate': 1500}, {'crop_id': 'C003', 'crop_name': 'Rice', 'soil_moisture_level': 50, 'temperature': 30, 'yield_estimate': 2000}]
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task Description #8: Airport Flight Management System An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

- Use AI to recommend algorithms.

- Justify the algorithm selection.
- Implement searching and sorting logic in Python.

```
def search_flight(flights, flight_id):
    """
    Searches for flight details by flight ID using linear search.

    Time Complexity: O(n) - where n is the number of flights.

    Parameters:
    flights (list): A list of flight records (dictionaries).
    flight_id: The ID of the flight to search for.

    Returns:
    dict: The flight record if found, otherwise None.
    """
    for flight in flights:
        if flight['flight_id'] == flight_id:
            return flight
    return None

def sort_flights(flights, key):
    """Sorts flights based on a specified key (departure time or arrival time) using the built-in sorted function.
    Time Complexity: O(n log n) - due to the sorting algorithm used by sorted().
    Parameters:
    flights (list): A list of flight records (dictionaries).
    key (str): The key to sort by ('departure_time' or 'arrival_time').

    Returns:
    list: A new list of sorted flight records.
    """
    if key not in ['departure_time', 'arrival_time']:
        raise ValueError("Invalid key: must be 'departure_time' or 'arrival_time'.")

    return sorted(flights, key=lambda x: x[key])

# Example usage
if __name__ == "__main__":
    flights = [
        {'flight_id': 'F001', 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'},
        {'flight_id': 'F002', 'airline_name': 'Airline B', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'Delayed'},
        {'flight_id': 'F003', 'airline_name': 'Airline C', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'On Time'}
    ]
```

```
# Search for a flight
flight = search_flight(flights, 'F002')
print("Search Result:", flight)

# Sort flights by departure time
sorted_by_departure = sort_flights(flights, key='departure_time')
print("Flights sorted by departure time:", sorted_by_departure)

# Sort flights by arrival time
sorted_by_arrival = sort_flights(flights, key='arrival_time')
print("Flights sorted by arrival time:", sorted_by_arrival)
```

```
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_12.5.py
Search Result: {'flight_id': 'F002', 'airline_name': 'Airline B', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'Delayed'}
Flights sorted by departure time: [{'flight_id': 'F003', 'airline_name': 'Airline C', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'On Time'}, {'flight_id': 'F001', 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'}, {'flight_id': 'F002', 'airline_name': 'Airline B', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'Delayed'}]
Flights sorted by arrival time: [{'flight_id': 'F003', 'airline_name': 'Airline C', 'departure_time': '2024-07-01 09:00', 'arrival_time': '2024-07-01 11:00', 'status': 'On Time'}, {'flight_id': 'F001', 'airline_name': 'Airline A', 'departure_time': '2024-07-01 10:00', 'arrival_time': '2024-07-01 12:00', 'status': 'On Time'}, {'flight_id': 'F002', 'airline_name': 'Airline B', 'departure_time': '2024-07-01 11:00', 'arrival_time': '2024-07-01 13:00', 'status': 'Delayed'}]
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>
```